GitOps

Git says what should run; the cluster pulls it and keeps it that way

Watch the 4-min video

In one line: in GitOps the desired state of every environment lives in Git, and an agent inside the cluster (Argo CD, Flux) **pulls** it and **continuously reconciles** the cluster to match, so deployments are pull requests, drift is corrected automatically, and rollback is a `git revert`.

1 The deploy that undid a fix

The CI pipeline has admin credentials to every cluster and deploys with `kubectl apply`. During an incident someone fixes production by editing a Deployment by hand. Git no longer describes what's running, nobody notices, and the next routine deploy quietly removes the fix. Two problems: **powerful credentials outside the cluster**, and **no one checks** that the live state still matches the declared state.

2 Four principles

The OpenGitOps project (CNCF) defines GitOps with four principles:

1. **Declarative**: the system's desired state is expressed declaratively (Kubernetes manifests, Helm values, Kustomize overlays).
2. **Versioned and immutable**: that state is stored with full history, in practice Git, so every change has an author, a review and a way back.
3. **Pulled automatically**: software agents fetch the desired state; nobody pushes it into the cluster.
4. **Continuously reconciled**: agents keep comparing actual with desired state and correct the difference, not just once at deploy time.

3 Pull, not push

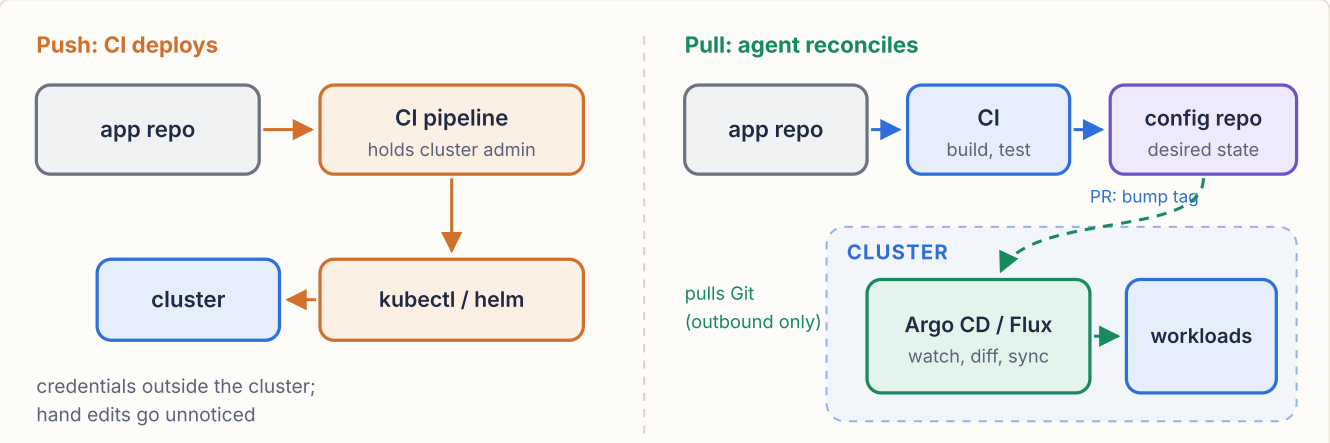


Figure 1. Push vs pull. In GitOps, CI never touches the cluster: it builds images and proposes changes to the config repo.

Argo CD or Flux runs **inside** each cluster (or in a management cluster) and needs only **read** access to Git and the image registry, through outbound connections. CI's job shrinks to testing and building images, then updating a tag in Git. Cluster credentials never leave the cluster, which removes the most valuable

secret from your CI system. A single central Argo CD managing many clusters is also common; it is easier to operate but holds credentials for all of them, so protect it like the old CI.

4 Reconciliation and drift

The agent re-reads Git on a timer (Argo CD's default is about every 3 minutes; a Git webhook makes it near-instant) and compares the rendered manifests with the live objects. Each application is **Synced** or **OutOfSync**, and separately **Healthy**, **Progressing** or **Degraded** (pods ready? rollout finished?).

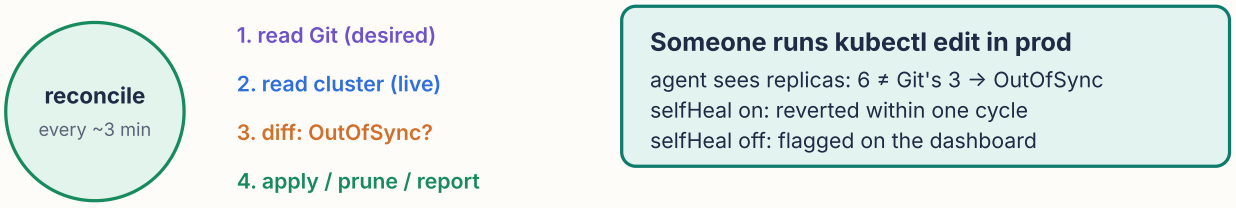


Figure 2. The reconcile loop. With self-heal, the hand edit from the opening story is undone within minutes instead of silently surviving until the next deploy.

The fix for the opening story is not "never edit by hand" but "hand edits don't last": during an incident you commit the fix to Git (fast, reviewed by one person), and the agent applies it. With `prune` enabled, objects deleted from Git are deleted from the cluster too, so nothing lingers.

5 Promotion and rollback through pull requests

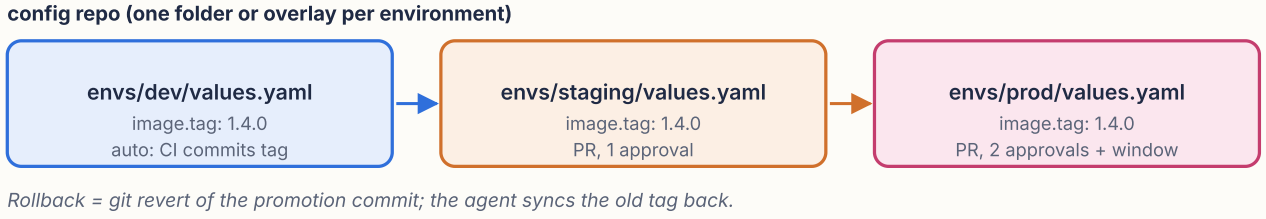


Figure 3. Promotion is a change of one line per environment, and the repository's branch protection becomes your deployment approval process.

CI writes the new image tag into the dev environment automatically (directly, or via Argo CD Image Updater / Flux image automation). Promoting to staging and production is a pull request that changes the tag in those folders; required reviewers, status checks and CODEOWNERS on `envs/prod/` are the approval gate, and the Git log is the audit trail of who deployed what, when. **Rollback** is reverting the commit. Use immutable tags or image digests, never `latest`, or Git can't say exactly what's running.

6 Caveats

- **Secrets** can't sit in Git in plain text. Options: **Sealed Secrets** (encrypted with the cluster's public key; only the in-cluster controller can decrypt), **SOPS** (encrypted values in files, decrypted by Flux or a plugin), or the **External Secrets Operator** (Git stores only a reference; the operator fetches the value from Key Vault, Secrets Manager or Vault). The last keeps secrets out of Git entirely and is the most common choice.
- **CI is still needed**: GitOps replaces only the deployment step, not tests, builds and scans.
- **Two repos** are common: application code, and environment configuration. This keeps config history clean and avoids CI loops where a tag bump triggers another build.
- **Ordering and migrations**: database migrations and dependent resources need sync waves or hooks (Argo CD) or `dependsOn` (Flux).

- **Progressive delivery** (canary, blue-green with automatic analysis) is a layer on top: Argo Rollouts or Flagger.
- GitOps fits declarative, Kubernetes-style resources best; cloud resources can join through Crossplane or Terraform controllers, but many teams keep those in an IaC pipeline (lesson 61).

WORKED EXAMPLE: ARGO CD SYNCING HELM CHARTS

A config repository holds a Helm chart per service and a values file per environment. Argo CD in each cluster has an `Application` pointing at its environment's values (below), with automated sync, prune and self-heal. CI builds **orders 1.4.0**, pushes the image, and opens a PR changing `image.tag` in `envs/prod/orders.yaml`. Two reviewers approve; the PR merges. The Git webhook notifies Argo CD, which renders the chart, sees one changed Deployment, and syncs it within a minute. The dashboard shows Progressing while new pods roll out, then Healthy. If error rates rise, the on-call engineer reverts the merge commit and Argo CD rolls back to 1.3.2.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: orders-prod, namespace: argocd }
spec:
  source:
    repoURL: https://git.example.com/platform/config.git
    path: charts/orders
    targetRevision: main
    helm: { valueFiles: [ ../../envs/prod/orders.yaml ] }
  destination: { server: https://kubernetes.default.svc, namespace: orders }
  syncPolicy:
    automated: { prune: true, selfHeal: true }   # delete removed objects, undo drift
```

Figure 4. An Argo CD Application: which Git path to render, where to deploy it, and whether to prune and self-heal.

COMMON MISTAKES

- Keeping CI's cluster credentials "just in case", so push deploys continue beside GitOps.
- Mutable tags like `latest`: Git doesn't change, so nothing deploys, or everything does.
- Plain Kubernetes Secrets committed to the config repo (base64 is not encryption).
- Self-heal off and nobody watching OutOfSync alerts: drift returns.

7 Key terms

Desired state: what Git declares should be running

Drift: live state differs from Git

Promotion: moving a version to the next environment via PR

Reconciliation: loop that makes live state match desired state

Sync / prune / self-heal: apply changes / delete removed objects / undo manual edits

Sealed / External Secrets: ways to keep secret values out of plain Git

8 Interview questions

Q1. How does GitOps differ from push-based CI/CD? BASIC

In push-based CI/CD the pipeline holds cluster credentials and applies changes; in GitOps Git declares the desired state and an in-cluster agent pulls and continuously reconciles it, correcting drift.

Q2. How do you promote and roll back? BASIC

With pull requests that change the environment's configuration (e.g. the image tag), and by reverting the commit to roll back.

Q3. Name the four OpenGitOps principles. BASIC

Declarative, versioned and immutable, pulled automatically, continuously reconciled.

Q4. Production is down and the fix is one config value. Do you kubectl edit? SCENARIO

Commit the change to Git and let the agent sync (or sync manually from Argo CD); a direct edit would be reverted by self-heal or lost at the next deploy.

Q5. How do you handle database passwords with GitOps? SCENARIO

Keep only references in Git and use the External Secrets Operator to pull values from a secret manager, or commit encrypted Sealed Secrets/SOPS files.

Q6. Why might CI loop forever in a single-repo GitOps setup? TRICKY

CI's commit of the new image tag triggers CI again; separate config repos or path filters and skip-CI rules prevent it.

Q7. Does GitOps remove the need for CI? TRICKY

No: CI still tests, scans and builds images and proposes the config change; GitOps replaces only how changes reach the cluster.

Next: SD-V-013 · Immutable Infrastructure backendroadmap.com